ML Integration Guide

Road Disruption Intelligence - how to add machine learning to this repository

Repository: **github.com/AnandRawat138/road-disruption-intelligence** (branch **main**)
Prepared for: the ML team working with Aviral Consulting (OPC) Private Limited
Codebase version: v1.0.0, 11 October 2026

How to use this document. Upload this PDF to your Claude or ChatGPT project together with access to the repository (or the relevant files). Section 15 contains a ready-to-paste instruction for the AI assistant. Sections 2-4 explain the code and the data; sections 5-12 say exactly **what** to build, **where** to put it and **how** to wire it in; sections 13-14 define the tests and the acceptance criteria.

Section	Contents
1	Goals and ground rules
2	Repository map - where everything lives
3	How a record flows through the system today
4	The data and the label situation (read before training anything)
5	Which ML tasks to build, in priority order
6	Target architecture: the rdi.services.ml package
7	Database changes and migration
8	Task 1 - Relevance model (triage)
9	Task 2 - Category model (cross-check)
10	Task 3 - Embeddings for duplicates and incident linking
11	API, settings, background jobs and user interface
12	Dependencies, Windows setup and model artifacts
13	Evaluation protocol and model card
14	Tests and definition of done
15	Ready-to-paste prompt for Claude / ChatGPT
16	Pitfalls to avoid

1. Goals and ground rules

The application already classifies news articles with a large language model (LLM) that the client controls (local Ollama or a cloud key). That works but is slow on local hardware (roughly 10-17 seconds per article on a 6 GB GPU) and about 14,800 articles are still unclassified. Machine learning should make the system **faster, cheaper and more consistent** - not replace human review.

Primary goals for the ML team

- A fast **relevance model** that scores every article in milliseconds so the LLM and the operators only see what matters.
- A **category model** that cross-checks the LLM and flags disagreements for review.
- **Text embeddings** that improve duplicate detection and the linking of related reports into one incident.
- A repeatable **training, evaluation and deployment** path that the client can run on a Windows PC without a GPU.

Ground rules (non-negotiable - the existing code follows them and reviewers will check them)

- Do not fabricate labels, metrics or results. Every reported number must come from code that was actually run.
- ML output is **advisory**. It may prioritise, pre-fill and flag; it must never auto-approve a record or delete data.
- Never overwrite the LLM classification or a human correction. Store ML predictions in their own fields/tables with the model name and version.
- The application must keep working when ML is not installed or no model is trained (graceful degradation).
- No new network services, no GPU requirement, no heavy frameworks (no PyTorch/TensorFlow) unless clearly justified.
- All 56 existing tests must keep passing; new behaviour needs new tests that run without network access.
- Keep the original files in `data/` untouched; never commit secrets, the runtime database or model files larger than ~20 MB.

2. Repository map - where everything lives

```
road-disruption-intelligence/                (repository root)
  data/                                      historical news data, JSONL, read-only input
  README.md                                  install + quick user guide
  road-disruption-intel/
    setup_and_start.bat  start.bat  stop.bat  setup_llm.bat  diagnose.bat
    .env.example                             configuration template (.env is never committed)
    backend/
      requirements.txt                       runtime deps (FastAPI, SQLAlchemy, httpx, feedparser...)
      requirements-dev.txt                   + pytest
      rdi/                                   <-- THE APPLICATION (v1)
        config.py                            settings from env (.env), runtime paths
        main.py                              FastAPI app factory, routers, security middleware
        server.py / cli.py                   entry points (python -m rdi.server / rdi.cli ...)
        db/models.py                         SQLAlchemy ORM models (articles, classifications, incidents...)
        db/session.py                        engine, init_db(), SCHEMA_VERSION, MIGRATIONS, backups
        api/records.py                       /api/records (filters, sort, export, review, corrections)
        api/incidents.py                     /api/incidents, /api/map/incidents
        api/dashboard.py                     /api/dashboard/summary, /api/quality, /api/reports/summary
        api/config_api.py                    /api/llm/*, /api/settings/*, /api/system/*, classification trigger
        api/operations.py                    sources, imports, jobs, alerts, audit
        services/ingest.py                   Ingestor: dedup + insert of every new article  (ML HOOK #1)
        services/prefilter.py                keyword pre-filter (the thing the relevance model improves)
        services/classification/schema.py   LLM prompt, output schema, validation
        services/classification/classifier.py classify_article(), review_reasons()      (ML HOOK #2)
        services/incidents.py                incident linking: find_match()            (ML HOOK #3)
        services/jobs.py, job_handlers.py    background job runner and job types       (ML HOOK #4)
        services/settings_store.py           persisted settings with DEFAULTS
        services/llm/providers.py            Ollama / OpenAI-compatible / Anthropic clients
        services/geo/                        gazetteer + Nominatim geocoder
        services/connectors/                 news sources
      tests/                                 pytest suite (conftest.py creates a throw-away DB)
      app/, seed_*.py, roaddb.db             legacy v0 code - DO NOT USE, reference only
    frontend/
      src/app/App.tsx                        navigation (NAV array) and routes
      src/app/pages/*.tsx                    one file per page (Records.tsx, LLMSettings.tsx, ...)
      src/app/components/RecordDrawer.tsx    record detail panel
      src/app/api.ts, meta.ts                API client, labels
      dist/                                  pre-built UI (rebuild with: npm ci && npm run build)
    runtime/                                 created at runtime, gitignored: rdi.db, logs, backups, secrets
    docs/                                    USER_GUIDE.md, API.md, screenshots
```

Everything new for ML goes into **backend/rdi/services/ml/** (runtime inference), **backend/ml_training/** (offline training/evaluation scripts) and **backend/tests/test_5_ml.py**. Trained models go to **runtime/models/** (not committed). Small edits are needed in the hook files listed above.

3. How a record flows through the system today

Step	Code	What happens
1 Ingest	services/ingest.py: Ingestor.ingest()	Article arrives from a file import or a news connector. Deduplicated by canonical URL, external id, content hash and same-source title. Keyword pre-filter sets prefilter_label = LIKELY_RELEVANT / POSSIBLY_RELEVANT / UNLIKELY.
2 Queue	services/job_handlers.py: _classify()	Operator (or auto-classify) starts a 'classify' job. Records are selected by filters, usually prefilter = LIKELY_RELEVANT, newest first.
3 LLM	services/classification/classifier.py: classify_article()	LLM returns JSON (relevance, category, location, time, severity, evidence). Output is validated; a new Classification version is stored with provider/model/prompt_version.
4 Geo	services/geo/geocoder.py	City/locality resolved to an approximate centroid (gazetteer or Nominatim).
5 Review flags	classifier.review_reasons()	Low confidence, unverified evidence, unresolved location etc. set needs_review.
6 Incident	services/incidents.py: link_article() / find_match()	RELEVANT records are linked to an existing incident (same category + city + time window + matching road/locality) or create a new one.
7 Human review	api/records.py: /review, PATCH /classification	Operator approves/rejects (review_status) or corrects fields (new HUMAN classification version). Approval confirms the incident.

ML plugs into steps 1, 2, 5 and 6. It does not replace step 3 (the LLM extracts structured details that a small classifier cannot) and it never performs step 7.

4. The data and the label situation (read before training anything)

4.1 What is in the database

Table / field	Meaning for ML
articles.title, summary_text, body_text	Model input text. Historical and Google News records are mostly **headline only** (summary = headline + publisher).
articles.source_name, published_at, connector, language	Useful metadata features / split keys.
articles.prefilter_label/score	Keyword rule output. A baseline to beat, NOT a label.
articles.dataset_flags	Provenance from old files, e.g. prior_lexical_score, prior_eval_split. NOT labels.
classifications (is_current=1, origin='MODEL')	LLM output: relevance, category, severity, city... = **silver labels** (noisy, produced by llama3:8b).
classifications (origin='HUMAN')	Operator corrections = **gold labels** for the corrected fields.
articles.review_status = APPROVED / REJECTED	Operator decision on the current classification = **gold** relevance signal (APPROVED: the classification was right; REJECTED: wrong/irrelevant).
incidents + articles.incident_id	Groups of reports about the same event - useful for embedding / linking evaluation and for group-aware splits.

4.2 The honest label situation (11 Oct 2026)

Signal	Count	Use
Unique articles	15,100	Unlabelled pool
LLM classifications (current)	~350 and growing (job running)	Silver training labels
Operator APPROVED / REJECTED	0	Gold - must be created
HUMAN corrections	0	Gold - must be created
data/weak_labels file	8,914 rows	Contains NO labels (only constant 0.85 confidence) - do not use as labels
data/evaluation splits	6,239 / 1,337 / 1,338	No ground truth inside - only a partition of keyword-relevant articles

Consequence: there is currently no ground truth to measure accuracy. Before claiming any metric the team must create a **gold evaluation set**: at least 300-500 articles reviewed by operators in the application (Classification queue: Approve / Reject / Edit fields), sampled across time, sources, cities and pre-filter labels (include UNLIKELY articles, otherwise recall cannot be measured). Train on silver + gold-train, evaluate only on gold-test.

4.3 Exporting a training dataset

Add `backend/ml_training/export_dataset.py` that reads the SQLite database through the existing session (so paths come from `rdi.config.settings`) and writes JSONL to `runtime/ml/datasets/`:

```
# backend/ml_training/export_dataset.py  (run from backend/:  ..\.venv\Scripts\python -m ml_training.export_dataset)
import json
from sqlalchemy import select
from rdi.config import settings
from rdi.db.session import SessionLocal
from rdi.db.models import Article, Classification
from rdi.services.classification.classifier import article_source_text

def label_source(a: Article, c: Classification | None) -> str:
    if c is not None and c.origin == "HUMAN":
        return "gold_corrected"
    if a.review_status in ("APPROVED", "REJECTED"):
        return "gold_reviewed"
    return "silver_llm" if c is not None else "unlabelled"

def main():
    out = settings.runtime_dir / "ml" / "datasets" / "articles.jsonl"
    out.parent.mkdir(parents=True, exist_ok=True)
    with SessionLocal() as db, open(out, "w", encoding="utf-8") as fh:
        rows = db.execute(select(Article, Classification).outerjoin(
            Classification, Classification.id == Article.current_classification_id)
            .where(Article.near_duplicate_of.is_(None)))
        for a, c in rows:
            relevance = c.relevance if c else None
            if a.review_status == "REJECTED":
                relevance = "IRRELEVANT"          # operator says the classification is wrong
            fh.write(json.dumps({
                "id": a.id, "text": article_source_text(a), "source": a.source_name,
                "published_at": a.published_at.isoformat() if a.published_at else None,
                "incident_id": a.incident_id, "prefilter": a.prefilter_label,
                "relevance": relevance, "category": c.category if c else None,
                "label_source": label_source(a, c),
                "classifier": f"{c.provider}/{c.model}/{c.prompt_version}" if c else None,
            }, ensure_ascii=False) + "\n")
    print("wrote", out)

if __name__ == "__main__":
    main()
```

Note on REJECTED: a rejection means the operator disagrees with the current classification. For relevance it is safest to treat it as 'not RELEVANT'; for category training, drop rejected rows rather than guessing the right category.

5. Which ML tasks to build, in priority order

#	Task	Why it matters	Model (first version)	Needs
1	Relevance triage	Score all 15k+ articles instantly; classify the right ones first; cut LLM time	TF-IDF (word + char n-grams) + Logistic Regression, calibrated	Silver labels now; gold test set to evaluate
2	Category cross-check	Catch LLM category mistakes (e.g. OTHER for diversions) and flag them	Same pipeline, multinomial	Silver + gold; >= 30 examples per category
3	Embeddings for duplicates & incident linking	Link differently-worded reports of one event; suggest duplicate incidents	Ollama nomic-embed-text (768-d, already installed) + cosine similarity	No training; tune threshold on reviewed incidents
4 (later)	Severity / time-status	Better prioritisation	Only after enough gold labels exist	Gold labels
5 (later)	Location NER	More records on the map	Gazetteer + rules first; spaCy only if justified	Annotated spans

Not in scope: training or fine-tuning LLMs, deep-learning pipelines needing a GPU, or replacing the LLM extractor. Start with the simplest model that beats the keyword pre-filter on the gold test set.

6. Target architecture: the rdi.services.ml package

```
backend/rdi/services/ml/
  __init__.py
  registry.py        load/save model artifacts, list available models, active version per task
  features.py        build model input text from an Article (reuse article_source_text)
  relevance.py       predict_relevance(texts) -> list[(label, probability)]
  category.py        predict_category(texts)  -> list[(category, probability)]
  embeddings.py      embed(texts) via Ollama /api/embed, cache in article_embeddings table
  scoring.py         score_article(db, article): runs enabled models, writes ml_predictions rows
backend/ml_training/                          (offline; not imported by the server)
  export_dataset.py   section 4.3
  train_relevance.py  train + evaluate + save artifact to runtime/models/relevance/<version>/
  train_category.py
  evaluate.py         metrics on the gold test set, writes the model card
backend/tests/test_5_ml.py
road-disruption-intel/docs/ML_MODEL_CARD.md   generated/updated by evaluate.py (real numbers only)
```

6.1 Artifact layout and registry

```
runtime/models/relevance/2026-10-20_tfidf_lr_v1/
    model.joblib         sklearn Pipeline (vectorizer + classifier + calibration)
    metadata.json        {"task": "relevance", "version": "2026-10-20_tfidf_lr_v1",
                          "created_at": "...", "trained_on": {"silver": 2300, "gold": 240},
                          "labels": ["RELEVANT", "NOT_RELEVANT"], "threshold": 0.42,
                          "metrics_gold_test": {"precision": ..., "recall": ..., "f1": ...},
                          "code_commit": "<git sha>", "sklearn_version": "1.5.2"}
runtime/models/active.json   {"relevance": "2026-10-20_tfidf_lr_v1", "category": null}
```

```
# backend/rdi/services/ml/registry.py  (sketch)
import json, logging, threading
from pathlib import Path
from ...config import settings
log = logging.getLogger(__name__)
_lock, _cache = threading.Lock(), {}

def models_dir() -> Path:
    return settings.runtime_dir / "models"

def ml_available() -> bool:
    try:
        import sklearn, joblib  # noqa: F401  (optional dependency, see section 12)
        return True
    except ImportError:
        return False

def active_version(task: str) -> str | None:
    f = models_dir() / "active.json"
    return json.loads(f.read_text()).get(task) if f.exists() else None

def load(task: str):
    """Return (model, metadata) for the active version, or (None, None). Never raises."""
    version = active_version(task)
    if not version or not ml_available():
        return None, None
    with _lock:
        if (task, version) not in _cache:
            import joblib
            d = models_dir() / task / version
            try:
                _cache[(task, version)] = (joblib.load(d / "model.joblib"),
                                           json.loads((d / "metadata.json").read_text()))
            except Exception:
                log.exception("Could not load ML model %s/%s", task, version)
                return None, None
        return _cache[(task, version)]
```

joblib/pickle files execute code when loaded. Only load model files produced by your own training scripts from runtime/models; never accept model uploads through the web API.

7. Database changes and migration

Important: `Base.metadata.create_all()` in `db/session.py` creates **new tables** but never adds **columns** to existing tables. Prefer new tables (below). If you must add a column to an existing table, bump `SCHEMA_VERSION` and register a migration function - init_db() backs up the database before running it.

Add two new tables to `backend/rdi/db/models.py` (they are created automatically on next start):

```
class MLPrediction(Base):
    """Advisory prediction of one model for one article. Never overwrites LLM/human output."""
    __tablename__ = "ml_predictions"
    __table_args__ = (UniqueConstraint("article_id", "task", "model_version", name="uq_ml_pred"),)
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    article_id: Mapped[str] = mapped_column(String(32), ForeignKey("articles.id", ondelete="CASCADE"), index=True)
    task: Mapped[str] = mapped_column(String(30), index=True)          # relevance / category
    model_version: Mapped[str] = mapped_column(String(80))
    label: Mapped[str | None] = mapped_column(String(30), index=True)
    probability: Mapped[float | None] = mapped_column(Float)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)

class ArticleEmbedding(Base):
    __tablename__ = "article_embeddings"
    article_id: Mapped[str] = mapped_column(String(32), ForeignKey("articles.id", ondelete="CASCADE"), primary_key=Tr
    model: Mapped[str] = mapped_column(String(80), primary_key=True)   # e.g. ollama/nomic-embed-text
    dim: Mapped[int] = mapped_column(Integer)
    vector: Mapped[bytes] = mapped_column(LargeBinary)                 # float32 little-endian
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
```

Optional fast filter column (only if querying ml_predictions is too slow): add `articles.ml_relevance_score` via a migration:

```
# backend/rdi/db/session.py
SCHEMA_VERSION = 2

def _m2_ml_columns(eng):
    with eng.begin() as con:
        cols = {r[1] for r in con.exec_driver_sql("PRAGMA table_info(articles)")}
        if "ml_relevance_score" not in cols:
            con.exec_driver_sql("ALTER TABLE articles ADD COLUMN ml_relevance_score FLOAT")
            con.exec_driver_sql("CREATE INDEX IF NOT EXISTS ix_articles_ml_rel ON articles (ml_relevance_score)")

MIGRATIONS: dict[int, callable] = {2: _m2_ml_columns}
# ...and add  ml_relevance_score: Mapped[float | None] = mapped_column(Float)  to Article in models.py
```

Update `DATA_DICTIONARY.md` for every new table/column.

8. Task 1 - Relevance model (triage)

8.1 Training (offline)

```
# backend/ml_training/train_relevance.py  (sketch - keep it this simple for v1)
import json, datetime, subprocess
from pathlib import Path
import joblib, numpy as np
from sklearn.pipeline import Pipeline, FeatureUnion
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import precision_recall_fscore_support, average_precision_score
from rdi.config import settings

rows = [json.loads(l) for l in open(settings.runtime_dir / "ml/datasets/articles.jsonl", encoding="utf-8")]
lab = [r for r in rows if r["relevance"]]
y = lambda r: 1 if r["relevance"] == "RELEVANT" else 0
gold = [r for r in lab if r["label_source"].startswith("gold")]
# time-based, incident-grouped split: newest 30% of GOLD = test; never put one incident on both sides
gold.sort(key=lambda r: r["published_at"] or "")
cut = int(len(gold) * 0.7)
test = gold[cut:]
test_groups = {r["incident_id"] for r in test if r["incident_id"]}
train = [r for r in lab if r not in test and (r["incident_id"] is None or r["incident_id"] not in test_groups)]

features = FeatureUnion([
    ("word", TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)),
    ("char", TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5), min_df=3, sublinear_tf=True)),
])
clf = CalibratedClassifierCV(LogisticRegression(max_iter=2000, class_weight="balanced"), cv=3)
pipe = Pipeline([("features", features), ("clf", clf)])
pipe.fit([r["text"] for r in train], [y(r) for r in train])

p = pipe.predict_proba([r["text"] for r in test])[:, 1]
# choose threshold for high RECALL (triage must not drop real disruptions), e.g. recall >= 0.95 on gold test
...  # compute precision/recall/F1/average precision, compare with the keyword pre-filter baseline
version = datetime.date.today().isoformat() + "_tfidf_lr_v1"
out = settings.runtime_dir / "models/relevance" / version
out.mkdir(parents=True, exist_ok=True)
joblib.dump(pipe, out / "model.joblib")
(out / "metadata.json").write_text(json.dumps({...}, indent=2))   # include REAL metrics + counts + git sha
```

If there is no gold test set yet, the script must say so in metadata ("metrics_gold_test": null, "evaluation": "silver only - not a measure of accuracy") and the UI must show that warning.

8.2 Inference (online)

```
# backend/rdi/services/ml/relevance.py
from . import registry
def predict_relevance(texts: list[str]) -> tuple[str | None, list[float]]:
    model, meta = registry.load("relevance")
    if model is None:
        return None, []
    return meta["version"], [float(p) for p in model.predict_proba(texts)[:, 1]]
```

8.3 Where it is wired in

Hook	File / function	Change
New articles	services/ingest.py: Ingestor.ingest()	Do NOT call the model per row inside ingest (keeps imports fast and transactions short). Instead collect new article ids and let the caller enqueue an 'ml_score' job (acquisition.run_source already collects new_ids; importer can do the same at batch end).
Backfill	services/job_handlers.py: new @handler("ml_score")	Score articles in batches of 500 (one predict_proba call per batch), write MLPrediction rows (+ ml_relevance_score), commit per batch, honour ctx.cancelled().
Queue priority	job_handlers.select_articles_for_classification()	Allow sort by ml_relevance_score desc and filter ml_relevance_min.
Auto-classify	acquisition._maybe_auto_classify()	Optional setting: classify new articles whose ML score >= threshold (in addition to / instead of the keyword pre-filter).
Records API	api/records.py: apply_record_filters(), SORTS, record_row()	Add filter ml_relevance_min, sort key ml_relevance, field ml_relevance in rows and exports.

```
# backend/rdi/services/job_handlers.py  (sketch)
@handler("ml_score")
def _ml_score(ctx: JobContext, params: dict) -> dict:
    from .ml.scoring import score_batch
    with SessionLocal() as db:
        ids = params.get("article_ids") or list(db.scalars(select(Article.id)))
    done = 0
    for i in range(0, len(ids), 500):
        if ctx.cancelled():
            break
        with SessionLocal() as db:
            score_batch(db, ids[i:i + 500])      # writes MLPrediction rows, no LLM calls
            db.commit()
        done += len(ids[i:i + 500])
        ctx.update(processed=done, total=len(ids), message=f"{done}/{len(ids)} scored")
    return {"message": f"Scored {done} article(s)"}
```

9. Task 2 - Category model (cross-check)

Train the same pipeline with the category as target on RELEVANT rows only (merge rare categories into OTHER for training if they have fewer than ~30 examples). Use it in two places:

- **Disagreement flag:** in `classifier.review_reasons()` add a reason when the ML model's top category differs from the LLM's category and the ML probability is high (e.g. >= 0.8): *"ML model suggests DIVERSION (0.86) - LLM said OTHER"*. Gate it behind a setting.
- **Suggestion in the record panel:** show 'ML suggestion: DIVERSION (86%)' in `RecordDrawer.tsx`; the operator applies it with Edit fields. Never change the category automatically.

```
# backend/rdi/services/classification/classifier.py  inside review_reasons(), after existing checks
ml = settings_store.get(db, "ml") if db is not None else {}   # pass db in, or read ccfg["ml_cross_check"]
if cls.relevance == "RELEVANT" and ml.get("category_cross_check"):
    from ..ml.category import predict_category
    version, preds = predict_category([article_source_text(article)])
    if preds:
        label, prob = preds[0]
        if label != cls.category and prob >= ml.get("cross_check_min_prob", 0.8):
            reasons.append(f"ML model suggests {label} ({prob:.0%}) - LLM said {cls.category}")
```

review_reasons() currently has the signature (cls, article, ccfg, issues). The cleanest change is to read the ML settings into ccfg in classify_article() before calling it, so the function signature stays pure.

10. Task 3 - Embeddings for duplicates and incident linking

Ollama is already installed on the client PC with **nomic-embed-text** (verified: returns 768-dimension vectors via POST http://127.0.0.1:11434/api/embed). Using it avoids adding PyTorch.

```
# backend/rdi/services/ml/embeddings.py  (sketch)
import struct, httpx
from ..settings_store import get as get_setting

def embed(db, texts: list[str], client: httpx.Client | None = None) -> list[list[float]]:
    cfg = get_setting(db, "ml")
    base = get_setting(db, "llm")["providers"]["ollama"]["base_url"]
    c = client or httpx.Client(timeout=60)
    r = c.post(f"{base}/api/embed", json={"model": cfg.get("embedding_model", "nomic-embed-text"), "input": texts})
    r.raise_for_status()
    return r.json()["embeddings"]

def pack(v: list[float]) -> bytes:   return struct.pack(f"<{len(v)}f", *v)
def unpack(b: bytes) -> list[float]: return list(struct.unpack(f"<{len(b)//4}f", b))
```

Where to use it

- **Incident linking** - `services/incidents.py: find_match()`. Today a candidate matches only if category + city + time window agree AND specific roads/localities agree. Add an optional similarity check: compute cosine similarity between the new article and the incident's linked articles; require similarity >= a tuned threshold (e.g. 0.80) **in addition to** category/city/time. Embeddings must only make linking stricter or resolve 'not comparable' cases - never merge different cities or categories.

- **Duplicate incident suggestions** - `api/dashboard.py: data_quality()` currently suggests pairs by category + city + 48 h. Add the similarity score and sort by it.
- **Near-duplicate articles** - optional: same event, different publisher is NOT a duplicate (keep both as evidence); only same-source near-identical texts may be marked near_duplicate_of.

Tune thresholds on incidents operators have reviewed (merged / detached actions are recorded in audit_log). Embedding calls are slow-ish on CPU: compute them in the 'ml_score' job, cache in article_embeddings, and never block an API request on them. If Ollama is unreachable, skip silently and keep current behaviour.

11. API, settings, background jobs and user interface

11.1 Settings - services/settings_store.py

```
DEFAULTS["ml"] = {
    "enabled": False,                    # off until a model is trained and evaluated
    "relevance_auto_classify_min": None, # e.g. 0.6 to auto-classify new articles above this score
    "category_cross_check": False,
    "cross_check_min_prob": 0.8,
    "embedding_model": "nomic-embed-text",
    "use_embeddings_for_linking": False,
    "link_similarity_min": 0.80,
}
# and add "ml" to EDITABLE_SETTING_KEYS in api/config_api.py
```

11.2 New API router - backend/rdi/api/ml_api.py (include it in main.py's router list)

Method / path	Purpose
GET /api/ml/status	ML installed?, active model per task, metadata (version, trained_on counts, gold metrics or the 'no gold evaluation' warning), last scoring job
POST /api/ml/score	Start 'ml_score' job: {article_ids?} or all unscored articles; returns job_id
POST /api/ml/embed	Start embedding job (optional)
POST /api/ml/activate	{task, version} - switch active model (writes runtime/models/active.json, audit_log entry)
GET /api/records?ml_relevance_min=0.7&sort=ml_relevance	Filter/sort records by ML score (extend api/records.py)

Do NOT expose training through the web API in v1 - training is an offline, reviewed step run from the command line.

11.3 Frontend

- `src/app/pages/LLMSettings.tsx` (or a new `pages/MLModels.tsx` + an entry in the NAV array in `App.tsx`): show ML status, active models with their metrics and warnings, buttons 'Score all articles' (uses JobProgress component), activate version, toggles for the ml settings.
- `src/app/pages/Records.tsx`: add an 'ML relevance' column (percentage badge) and a 'Min ML score' filter; include the param in the URL like the other filters (useFilterParams LIST_KEYS/SCALAR_KEYS).
- `src/app/components/RecordDrawer.tsx`: show 'ML relevance 0.91 (model v...)' and 'ML category suggestion' under Classification, clearly labelled as advisory.
- `src/app/pages/Overview.tsx`: optional tile 'Unclassified with high ML score' linking to the filtered records list.
- Rebuild the UI after changes: `cd frontend && npm ci && npm run build` and commit `frontend/dist` (the client has no Node.js).

12. Dependencies, Windows setup and model artifacts

```
# backend/requirements-ml.txt   (optional extra - NOT added to requirements.txt)
-r requirements.txt
scikit-learn>=1.5,<2.0
joblib>=1.4
numpy>=1.26
```

- Install with `..\.venv\Scripts\python -m pip install -r requirements-ml.txt`. Verify wheels exist for Python 3.11-3.14 on Windows before pinning versions.
- Add an optional `setup_ml.bat` (copy the style of setup_llm.bat): install requirements-ml.txt, run export_dataset, train_relevance, evaluate, then print the model card summary and ask before activating the model.
- The server must start without these packages: import sklearn/joblib lazily (registry.ml_available()).
- `runtime/` is already gitignored, so trained models are not committed. To ship a model to the client, attach it to a GitHub Release (zip of runtime/models//) - not to the repository.
- Record the scikit-learn version in metadata.json; refuse to load a model trained with a different major version.

13. Evaluation protocol and model card

- **Gold test set only** for reported metrics (operator-reviewed). Silver (LLM) agreement may be reported separately and must be labelled 'agreement with llama3:8b, not accuracy'.
- **Split by time** (train on older, test on newer) and keep all articles of one incident on the same side.
- **Baselines:** compare against (a) the keyword pre-filter (LIKELY_RELEVANT = positive) and (b) the LLM itself on the same gold test.
- **Relevance metrics:** precision, recall, F1, average precision; choose the operating threshold for recall >= 0.95 (triage must not hide real disruptions) and report the precision at that threshold.
- **Category metrics:** per-class precision/recall/F1, macro-F1, confusion matrix; report class counts.
- **Linking:** pairwise precision/recall of 'same incident' decisions on reviewed incidents; report how many merges the embedding rule would have caught and how many wrong links it would add.
- **Speed:** articles scored per second on the client CPU.
- Write everything to `docs/ML_MODEL_CARD.md`: data counts, label sources, splits, metrics with confidence intervals (bootstrap), known failure modes, date, git commit. No hand-edited numbers.

14. Tests and definition of done

14.1 Tests - backend/tests/test_5_ml.py

The test suite uses a throw-away database (tests/conftest.py) and mocks all HTTP. ML tests must not need a GPU, Ollama or downloaded models. Mark them to skip cleanly (with a reason) if scikit-learn is not installed.

```
import pytest, httpx
sklearn = pytest.importorskip("sklearn", reason="ML extras not installed")

def test_registry_returns_none_without_model(): ...          # app works with no trained model
def test_train_tiny_relevance_model_and_score(db, tmp_path): ...  # train on ~20 fixture texts, save, load, score
def test_ml_score_job_writes_predictions_and_is_idempotent(db): ...
def test_ml_never_changes_llm_or_human_output(db): ...        # classification rows untouched
def test_category_disagreement_flag(db): ...                 # review_reasons adds the ML reason only when enabled
def test_embeddings_via_mocked_ollama(db): ...               # httpx.MockTransport for /api/embed
def test_linking_with_embeddings_never_merges_different_cities(db): ...
def test_ml_status_endpoint_reports_no_gold_evaluation(client): ...
def test_records_filter_ml_relevance_min(client): ...
```

Run: `cd backend && ..\.venv\Scripts\python -m pytest tests -q` (expected: 56 existing + new tests, 0 failed).

14.2 Definition of done

#	Criterion
1	All existing tests pass; new ML tests pass; frontend type check and build pass.
2	App starts and works with ML extras NOT installed and with no model trained.
3	A gold evaluation set of >= 300 operator-reviewed articles exists (or the gap is clearly documented).
4	docs/ML_MODEL_CARD.md generated from real runs, comparing ML vs keyword pre-filter vs LLM on the gold test set.
5	ML predictions stored with model version; LLM and human classifications never modified by ML.
6	Records list/export show ML score; filter and sort work server-side; UI labels ML as advisory.
7	'ml_score' job is batched, cancellable, resumable, and does not hold the DB write lock for long.
8	Embedding-based linking is behind a setting, tuned on reviewed incidents, and covered by tests.
9	README, USER_GUIDE (new section + screenshot), DATA_DICTIONARY, API.md, CHANGELOG and TEST_REPORT updated.
10	No secrets, runtime DB or model binaries committed; work done on a feature branch with a pull request.

15. Ready-to-paste prompt for Claude / ChatGPT

Upload this PDF to the project, give the assistant the repository (clone, zip or the files named in section 2), then paste:

```
You are working on the repository github.com/AnandRawat138/road-disruption-intelligence
(Road Disruption Intelligence, FastAPI + SQLAlchemy/SQLite backend in road-disruption-intel/backend/rdi,
React/TypeScript frontend in road-disruption-intel/frontend). The attached "ML Integration Guide" PDF is the
specification. Follow it exactly.

Task: implement the ML layer described in the guide, in this order, one pull request per milestone:
  M1  rdi/services/ml/ (registry, features, relevance), ml_training/export_dataset.py and
      train_relevance.py, MLPrediction table, "ml_score" job, /api/ml/status + /api/ml/score,
      requirements-ml.txt, tests/test_5_ml.py.
  M2  Records API + UI: ML score column, filter, sort, export field; ML status panel; record-drawer display.
  M3  Category model + disagreement review flag (behind the "ml" settings), tests.
  M4  Embeddings via Ollama nomic-embed-text, article_embeddings table, optional similarity check in
      incidents.find_match() and duplicate suggestions, tests.
  M5  evaluate.py + docs/ML_MODEL_CARD.md + documentation updates (README, USER_GUIDE, DATA_DICTIONARY,
      API.md, CHANGELOG, TEST_REPORT).

Rules:
- Read the existing code before changing it; match its style (type hints, small functions, docstrings
  where the surrounding code has them). Do not rewrite unrelated code.
- ML output is advisory: never modify Classification rows, review_status or incidents automatically
  based on ML; never auto-approve.
- The app must run without scikit-learn installed and without a trained model.
- No PyTorch/TensorFlow; CPU-only; Windows; Python 3.11-3.14.
- Do not invent labels or metrics. There are currently no gold labels: report that honestly and use
  silver (LLM) labels only for training, never as proof of accuracy.
- Keep SQLite transactions short (commit per batch); never hold a write transaction during slow work.
- All existing tests must pass; add tests that run without network (use httpx.MockTransport).
- Explain each change: file, function, why. At the end list what was run (tests, build) with real output.
Start with M1. First summarise the relevant existing code (ingest.py, job_handlers.py, classifier.py,
db/models.py, db/session.py) to confirm your understanding, then propose the exact diff.
```

16. Pitfalls to avoid

Pitfall	Why it is a problem / what to do instead
Using data/weak_labels as labels	It has no labels - only a constant 0.85 confidence.
Using the 'city' field from data/*.jsonl as location	It is the city that was searched for, not where the event happened (e.g. 'National Highway' appears as a city).
Evaluating on LLM labels and calling it accuracy	It only measures agreement with llama3:8b. Use the gold set.
Random train/test split	The same story appears from several publishers and queries; random splits leak. Split by time and by incident.
Dropping articles the keyword pre-filter calls UNLIKELY	Then recall can never be measured or improved. Sample them into the gold set.
Calling the model inside Ingestor.ingest() per row	Slows imports of 78k rows and lengthens transactions. Batch in a job.
Merging reports from different publishers as duplicates	They are independent evidence for one incident; link them, do not deduplicate them.
Overwriting the LLM category with the ML one	Breaks provenance and audit. Store separately, flag for review.
Shipping .joblib files in Git / loading untrusted pickles	Large binaries bloat the repo; pickles execute code. Use GitHub Releases and only your own artifacts.
Treating model probability as certainty	Calibrate, document the threshold, show it as advisory.
Long-running work in API requests	Use the job runner (services/jobs.py) like import/classify do.

Questions about the existing behaviour can be answered from the repository documentation: README.md, road-disruption-intel/DATA_DICTIONARY.md, docs/API.md, AUDIT_REPORT.md and TEST_REPORT.md.